# LAB PROGRAM

# CHAPTER 6

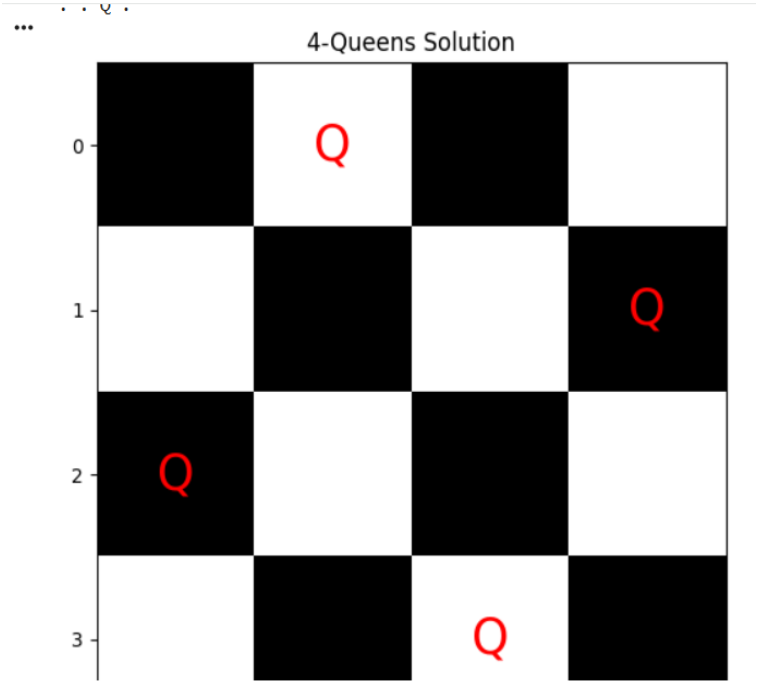
**1.Discuss the importance of visualizing the solutions of the N-Queens Problem to understand the placement of queens better. Use a graphical representation to show how queens are placed on the board for different values of N. Explain how visual tools can help in debugging the algorithm and gaining insights into the problem's complexity. Provide examples of visual representations for N = 4, N = 5, and N = 8, showing different valid solutions.**

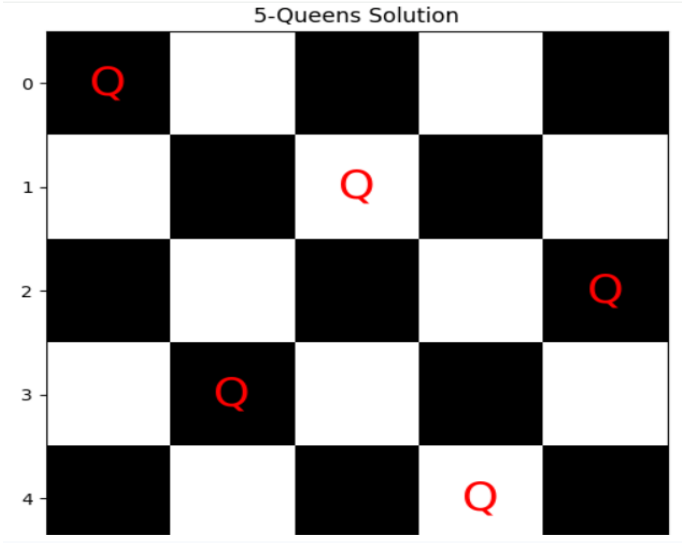
**CODE**

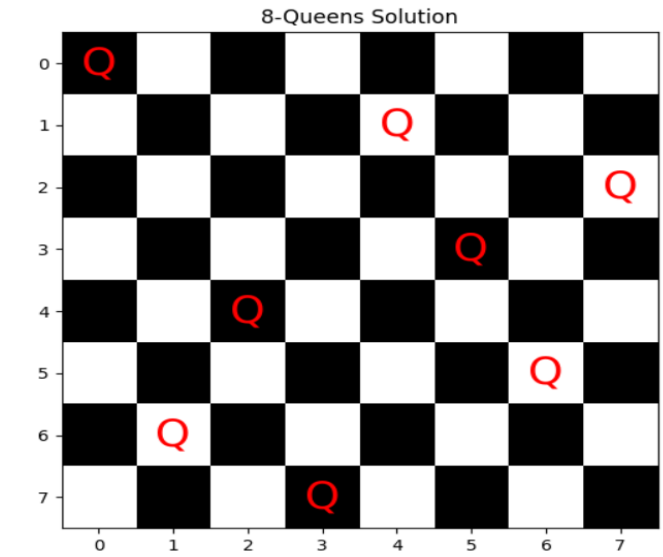
```
import matplotlib.pyplot as plt
def solve_n_queens(n):
    board=[[-1]*n for _ in range(n)]
    solutions=[]
    def safe(row,col):
        for i in range(row):
            if board[i][col]==1:
                return False
            if col-(row-i)>=0 and board[i][col-(row-i)]==1:
                return False
            if col+(row-i)<n and board[i][col+(row-i)]==1:
                return False
        return True
    def solve(row):
        if row==n:
            solutions.append([r[:] for r in board])
            return
        for col in range(n):
            if safe(row,col):
                board[row][col]=1
                solve(row+1)
                board[row][col]=-1
    solve(0)
    return solutions
def display_board(board,title):
```

```
    n=len(board)
    plt.figure(figsize=(6,6))
    plt.imshow([[0 if (i+j)%2==0 else 1 for j in range(n)] for i in range(n)],cmap="gray")
    for i in range(n):
        for j in range(n):
            if board[i][j]==1:
                plt.text(j,i,"Q",ha="center",va="center",fontsize=25,color="red")
    plt.xticks(range(n))
    plt.yticks(range(n))
    plt.title(title)
    plt.show()
for n in [4,5,8]:
    solutions=solve_n_queens(n)
    print("N =",n)
    for row in solutions[0]:
        print(" ".join("Q" if x==1 else "." for x in row))
    display_board(solutions[0],f"{n}-Queens Solution")
```

**OUTPUT**






**2. Discuss the generalization of the N-Queens Problem to other board sizes and shapes, such as rectangular boards or boards with obstacles. Explain how the algorithm can be adapted to handle these variations and the additional constraints they introduce. Provide examples of solving generalized N-Queens Problems for different board configurations, such as an 8×10 board, a 5×5 board with obstacles, and a 6×6 board with restricted positions.**

**CODE**

```
def solve_general(rows,cols,obstacles=set(),allowed=None):
    board=[[-1]*cols for _ in range(rows)]
    solution=[]
    def safe(row,col):
        if (row,col) in obstacles:
            return False
        if allowed is not None and col not in allowed[row]:
            return False
```

```
        for r,c in solution:
            if c==col or abs(r-row)==abs(c-col):
                return False
        return True
    def backtrack(row):
        if row==rows:
            return True
        for col in range(cols):
            if safe(row,col):
                solution.append((row,col))
                if backtrack(row+1):
                    return True
                solution.pop()
        return False
    if backtrack(0):
        return [c+1 for r,c in solution]
    return None
print("8x10 Board:",solve_general(8,10))
obstacles={(1,1),(3,3)}
print("5x5 Board with Obstacles:",solve_general(5,5,obstacles))
allowed=[set(range(6)) for _ in range(6)]
allowed[0]={0,2,4}
print("6x6 Board with Restricted Positions:",solve_general(6,6,set(),allowed))
```

**OUTPUT**

```
8x10 Board: [1, 3, 5, 2, 8, 10, 4, 7]
5x5 Board with Obstacles: [1, 3, 5, 2, 4]
6x6 Board with Restricted Positions: [3, 6, 2, 5, 1, 4]
```

**3. Write a program to solve a Sudoku puzzle by filling the empty cells.A sudoku solution must satisfy all of the following rules:Each of the digits 1-9 must occur exactly once in each row.Each of the digits 1-9 must occur exactly once in each column.Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub- boxes of the grid.The '.' character indicates empty cells.**

**CODE**

```
def solveSudoku(board):
    def valid(row,col,num):
```

```
        for i in range(9):
            if board[row][i]==num:
                return False
            if board[i][col]==num:
                return False
        sr=(row//3)*3
        sc=(col//3)*3
        for i in range(sr,sr+3):
            for j in range(sc,sc+3):
                if board[i][j]==num:
                    return False
        return True
    def solve():
        for i in range(9):
            for j in range(9):
                if board[i][j]==".":
                    for num in "123456789":
                        if valid(i,j,num):
                            board[i][j]=num
                            if solve():
                                return True
                            board[i][j]="."
                    return False
        return True
    solve()
board=[
["5","3",".",".","7",".",".",".",".","."],
["6",".",".","1","9","5",".",".",".","."],
[".","9","8",".",".",".",".",".","6","."],
["8",".",".",".",".","6",".",".",".","3"],
["4",".",".","8",".","3",".",".",".","1"],
["7",".",".",".",".","2",".",".",".","6"],
```

[".","6",".",".",".",".","2","8","."],

[".",".",".","4","1","9",".",".","5"],

[".",".",".",".","8",".",".","7","9"]

]

solveSudoku(board)

for row in board:

print(row)

**OUTPUT**

```
['5', '3', '4', '6', '7', '8', '9', '1', '2']
['6', '7', '2', '1', '9', '5', '3', '4', '8']
['1', '9', '8', '3', '4', '2', '5', '6', '7']
['8', '5', '9', '7', '6', '1', '4', '2', '3']
['4', '2', '6', '8', '5', '3', '7', '9', '1']
['7', '1', '3', '9', '2', '4', '8', '5', '6']
['9', '6', '1', '5', '3', '7', '2', '8', '4']
['2', '8', '7', '4', '1', '9', '6', '3', '5']
['3', '4', '5', '2', '8', '6', '1', '7', '9']
```

**4. Write a program to solve a Sudoku puzzle by filling the empty cells.A sudokusolution must satisfy all of the following rules:Each of the digits 1-9 must occur exactly once in each row.Each of the digits 1-9 must occur exactly once in each column.Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub- boxes of the grid.The '.' character indicates empty cells.**

**CODE**

```
def solveSudoku(board):
    def valid(row,col,num):
        for i in range(9):
            if board[row][i]==num:
                return False
            if board[i][col]==num:
                return False
        sr=(row//3)*3
        sc=(col//3)*3
        for i in range(sr,sr+3):
            for j in range(sc,sc+3):
                if board[i][j]==num:
                    return False
        return True
```

```
    def solve():
        for i in range(9):
            for j in range(9):
                if board[i][j]==".":
                    for num in "123456789":
                        if valid(i,j,num):
                            board[i][j]=num
                            if solve():
                                return True
                            board[i][j]="."
                    return False
        return True
    solve()
board=[
["5","3",".",".","7",".",".",".",".","."],
["6",".",".","1","9","5",".",".",".","."],
[".","9","8",".",".",".",".",".","6","."],
["8",".",".",".",".","6",".",".",".","3"],
["4",".",".","8",".","3",".",".",".","1"],
["7",".",".",".",".","2",".",".",".","6"],
[".","6",".",".",".",".",".","2","8","."],
[".",".",".","4","1","9",".",".",".","5"],
[".",".",".",".",".","8",".",".","7","9"]
]
solveSudoku(board)
for row in board:
    print(row)
```

**OUTPUT**

```
['5', '3', '4', '6', '7', '8', '9', '1', '2']
['6', '7', '2', '1', '9', '5', '3', '4', '8']
['1', '9', '8', '3', '4', '2', '5', '6', '7']
['8', '5', '9', '7', '6', '1', '4', '2', '3']
['4', '2', '6', '8', '5', '3', '7', '9', '1']
['7', '1', '3', '9', '2', '4', '8', '5', '6']
['9', '6', '1', '5', '3', '7', '2', '8', '4']
['2', '8', '7', '4', '1', '9', '6', '3', '5']
['3', '4', '5', '2', '8', '6', '1', '7', '9']
```

**5. You are given an integer array nums and an integer target. You want to build an expression out of nums by adding one of the symbols '+' and '-' before each integer in nums and then concatenate all the integers.For example, if nums = [2, 1], you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1" Return the number of different expressions that you can build, which evaluates to target.**

**CODE**

```
def findTargetSumWays(nums,target):
    dp={0:1}
    for num in nums:
        newdp={}
        for total,count in dp.items():
            newdp[total+num]=newdp.get(total+num,0)+count
            newdp[total-num]=newdp.get(total-num,0)+count
        dp=newdp
    return dp.get(target,0)
nums=[1,1,1,1,1]
target=3
print("Example 1 Output:",findTargetSumWays(nums,target))
nums=[1]
target=1
print("Example 2 Output:",findTargetSumWays(nums,target))
```

**OUTPUT**

```
Example 1 Output: 5
Example 2 Output: 1
```

**6. Given an array of integers arr, find the sum of min(b), where b ranges over every (contiguous) subarray of arr. Since the answer may be large, return the answer modulo 109 + 7.**

**CODE**

```
def sumSubarrayMins(arr):
    MOD=10**9+7
    n=len(arr)
    stack=[]
    left=[0]*n
    right=[0]*n
```

```
    for i in range(n):
        while stack and arr[stack[-1]]>arr[i]:
            stack.pop()
        if stack:
            left[i]=i-stack[-1]
        else:
            left[i]=i+1
        stack.append(i)
    stack=[]
    for i in range(n-1,-1,-1):
        while stack and arr[stack[-1]]>=arr[i]:
            stack.pop()
        if stack:
            right[i]=stack[-1]-i
        else:
            right[i]=n-i
        stack.append(i)
    ans=0
    for i in range(n):
        ans=(ans+arr[i]*left[i]*right[i])%MOD
    return ans
arr=[3,1,2,4]
print("Example 1 Output:",sumSubarrayMins(arr))
arr=[11,81,94,43,3]
print("Example 2 Output:",sumSubarrayMins(arr))
```

**OUTPUT**

```
Example 1 Output: 17
Example 2 Output: 444
```

**7. Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sumto target. You may return the combinations in any order.The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given**

**CODE**

```
def combinationSum(candidates,target):
    result=[]
    candidates.sort()
    def backtrack(start,target,path):
        if target==0:
            result.append(path[:])
            return
        for i in range(start,len(candidates)):
            if candidates[i]>target:
                break
            path.append(candidates[i])
            backtrack(i,target-candidates[i],path)
            path.pop()
    backtrack(0,target,[])
    return result
candidates=[2,3,6,7]
target=7
print("Example 1 Output:",combinationSum(candidates,target))
candidates=[2,3,5]
target=8
print("Example 2 Output:",combinationSum(candidates,target))
```

**OUTPUT**

```
Example 1 Output: [[2, 2, 3], [7]]
Example 2 Output: [[2, 2, 2, 2], [2, 3, 3], [3, 5]]
```

**8.Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target. Each number in candidates may only be used once in the combination. The solution set must not contain duplicate combinations.**

**CODE**

```
def combinationSum2(candidates,target):
    result=[]
```

```
    candidates.sort()
    def backtrack(start,target,path):
        if target==0:
            result.append(path[:])
            return
        for i in range(start,len(candidates)):
            if i>start and candidates[i]==candidates[i-1]:
                continue
            if candidates[i]>target:
                break
            path.append(candidates[i])
            backtrack(i+1,target-candidates[i],path)
            path.pop()
    backtrack(0,target,[])
    return result
candidates=[10,1,2,7,6,1,5]
target=8
print("Example 1 Output:",combinationSum2(candidates,target))
candidates=[2,5,2,1,2]
target=5
print("Example 2 Output:",combinationSum2(candidates,target))
```

**OUTPUT**

```
Example 1 Output: [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]
Example 2 Output: [[1, 2, 2], [5]]
```

**9. Given an array nums of distinct integers, return all the possible permutations. You can return the answer in any order.**

**CODE**

```
def permute(nums):
    result=[]
    def backtrack(path,used):
        if len(path)==len(nums):
            result.append(path[:])
```

```
            return
        for i in range(len(nums)):
            if not used[i]:
                used[i]=True
                path.append(nums[i])
                backtrack(path,used)
                path.pop()
                used[i]=False
    backtrack([], [False]*len(nums))
    return result
nums=[1,2,3]
print("Example 1 Output:",permute(nums))
nums=[0,1]
print("Example 2 Output:",permute(nums))
nums=[1]
print("Example 3 Output:",permute(nums))
```

**OUTPUT**

```
Example 1 Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
Example 2 Output: [[0, 1], [1, 0]]
Example 3 Output: [[1]]
```

**10. 10. Given a collection of numbers, nums, that might contain duplicates, return all possible unique permutations in any order.**

**CODE**

```
def permuteUnique(nums):
    result=[]
    nums.sort()
    def backtrack(path,used):
        if len(path)==len(nums):
            result.append(path[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            if i>0 and nums[i]==nums[i-1] and not used[i-1]:
```

```
            continue
        used[i]=True
        path.append(nums[i])
        backtrack(path,used)
        path.pop()
        used[i]=False
    backtrack([], [False]*len(nums))
    return result
nums=[1,1,2]
print("Example 1 Output:",permuteUnique(nums))
nums=[1,2,3]
print("Example 2 Output:",permuteUnique(nums))
```

**OUTPUT**

```
Example 1 Output: [[1, 1, 2], [1, 2, 1], [2, 1, 1]]
Example 2 Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

**11. You and your friends are assigned the task of coloring a map with a limited number of colors. The map is represented as a list of regions and their adjacency relationships. The rules are as follows: At each step, you can choose any uncolored region and color it with any available color. Your friend Alice follows the same strategy immediately after you, and then your friend Bob follows suit. You want to maximize the number of regions you personally color. Write a function that takes the map's adjacency list representation and returns the maximum number of regions you can color before all regions are colored. Write a program to implement the Graph coloring technique for an undirected graph. Implement an algorithm with minimum number of colors. edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] No. of vertices, n = 4**

**CODE**

```
def graph_coloring(n,edges):
    graph=[[] for _ in range(n)]
    for u,v in edges:
        graph[u].append(v)
        graph[v].append(u)
    color=[-1]*n
    def safe(v,c):
        for u in graph[v]:
            if color[u]==c:
                return False
        return True
    def solve(v):
```

```
        if v==n:
            return True
        for c in range(n):
            if safe(v,c):
                color[v]=c
                if solve(v+1):
                    return True
                color[v]=-1
        return False
    solve(0)
    return max(color)+1,color
edges=[(0,1),(1,2),(2,3),(3,0),(0,2)]
n=4
colors,coloring=graph_coloring(n,edges)
print("Minimum number of colors:",colors)
print("Vertex coloring:",coloring)
```

**OUTPUT**

```
Minimum number of colors: 3
Vertex coloring: [0, 1, 2, 1]
```

**12. You and your friends are tasked with coloring a map using a limited set ofcolors, with the following rules: At each step, you can choose any region of the map that hasn't been colored yet and color it with any available color. Yourfriend Alice will then color the next region using the same strategy, followed by your friend Bob. You aim to maximize the number of regions you color. Given a map represented as a list of regions and their adjacency relationships, write a function to determine the maximum number of regions you can color. Write a program to implement the Graph coloring technique for an undirected graph. Implement an algorithm with minimum number of colors. edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] No. of vertices, n = 4, k = 3**

**CODE**

```
def graph_coloring_k(n,edges,k):
    graph=[[] for _ in range(n)]
    for u,v in edges:
        graph[u].append(v)
        graph[v].append(u)
    color=[-1]*n
    def safe(v,c):
        for u in graph[v]:
```

```
            if color[u]==c:
                return False
        return True
    def solve(v):
        if v==n:
            return True
        for c in range(k):
            if safe(v,c):
                color[v]=c
                if solve(v+1):
                    return True
                color[v]=-1
        return False
    if solve(0):
        return True,color
    return False,None
edges=[(0,1),(1,2),(2,3),(3,0),(0,2)]
n=4
k=3
possible,coloring=graph_coloring_k(n,edges,k)
print("Can the graph be colored using",k,"colors?",possible)
if possible:
    print("Color assignment:",coloring)
```

**OUTPUT**

```
Can the graph be colored using 3 colors? True
Color assignment: [0, 1, 2, 1]
```

**13. You are given an undirected graph represented by a list of edges and the number of vertices n. Your task is to determine if there exists a Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex. Write a function that takes the list of edges and the number of vertices as input and returns true if there exists aHamiltonian cycle in the graph, otherwise return false. Example: Given edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2), (2, 4), (4, 0)] and n = 5**

**CODE**

```
def hamiltonian_cycle(n,edges):
    graph=[[] for _ in range(n)]
    for u,v in edges:
```

```
        graph[u].append(v)
        graph[v].append(u)
    path=[0]
    visited={0}
    def solve():
        if len(path)==n:
            return path[0] in graph[path[-1]]
        for v in graph[path[-1]]:
            if v not in visited:
                visited.add(v)
                path.append(v)
                if solve():
                    return True
                path.pop()
                visited.remove(v)
        return False
    if solve():
        return True,path+[0]
    return False,None
edges=[(0,1),(1,2),(2,3),(3,0),(0,2),(2,4),(4,0)]
n=5
possible,cycle=hamiltonian_cycle(n,edges)
print("Hamiltonian Cycle Exists:",possible)
if possible:
    print("Cycle:",cycle)
```

**OUTPUT**

```
•••  Hamiltonian Cycle Exists: False
```

**14. You are given an undirected graph represented by a list of edges and the number of vertices n. Your task is to determine if there exists a Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex. Write a function that takes the list of edges and the number of vertices as input and returns true if there exists a Hamiltonian cycle in the graph, otherwise return false. Example:edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] and n = 4**

**CODE**

```
def hamiltonian_cycle(n,edges):
    graph=[[] for _ in range(n)]
    for u,v in edges:
        graph[u].append(v)
        graph[v].append(u)
    path=[0]
    visited={0}
    def solve():
        if len(path)==n:
            return path[0] in graph[path[-1]]
        for v in graph[path[-1]]:
            if v not in visited:
                visited.add(v)
                path.append(v)
                if solve():
                    return True
                path.pop()
                visited.remove(v)
        return False
    return solve(),path+[0] if len(path)==n else []
edges=[(0,1),(1,2),(2,3),(3,0),(0,2)]
n=4
possible,cycle=hamiltonian_cycle(n,edges)
print("Hamiltonian Cycle Exists:",possible)
if possible:
    print("Cycle:",cycle)
```

**OUTPUT**

```
Hamiltonian Cycle Exists: True
Cycle: [0, 1, 2, 3, 0]
```

**15. You are tasked with designing an efficient coading to generate all subsets of a given set S containing n elements. Each subset should be outputted in lexicographical order. Return a list of lists where each inner list is a subset of the given set. Additionally, find out how your coading handles duplicateelements in S. A = [1, 2, 3] The subsets of [1, 2, 3] are: [], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]**

**CODE**

```
def subsets_lex(A):
    A=sorted(A)
    result=[]
    def backtrack(index,path):
        result.append(path[:])
        for i in range(index,len(A)):
            path.append(A[i])
            backtrack(i+1,path)
            path.pop()
    backtrack(0,[])
    result.sort(key=lambda x:[str(v) for v in x])
    return result
A=[1,2,3]
print("Subsets:")
for s in subsets_lex(A):
    print(s)
```

**OUTPUT**

```
Subsets:
[]
[1]
[1, 2]
[1, 2, 3]
[1, 3]
[2]
[2, 3]
[3]
```

**16. Write a program to implement the concept of subset generation. Given a set of unique integers and a specific integer 3, generate all subsets that contain the element 3. Return a list of lists where each inner list is a subset containing the element 3 E = [2, 3, 4, 5], x = 3, The subsets containing 3 : [3], [2, 3], [3, 4], [3,5], [2, 3, 4], [2, 3, 5], [3, 4, 5], [2, 3, 4, 5] Given an integer array nums of unique elements, return all possible subsets(the power set). The solution set must not contain duplicate subsets. Return the solution in any order.**

**CODE**

```
def subsets_containing(A,x):
    result=[]
    def backtrack(index,path):
        if index==len(A):
```

```
            if x in path:
                result.append(path[:])
            return
        backtrack(index+1,path)
        path.append(A[index])
        backtrack(index+1,path)
        path.pop()
    backtrack(0,[])
    return result
E=[2,3,4,5]
x=3
result=subsets_containing(E,x)
result.sort(key=lambda a:(len(a),a))
print("Subsets containing",x,":")
for s in result:
    print(s)
```

**OUTPUT**

```
Subsets containing 3 :
[3]
[2, 3]
[3, 4]
[3, 5]
[2, 3, 4]
[2, 3, 5]
[3, 4, 5]
[2, 3, 4, 5]
```

**17. You are given two string arrays words1 and words2. A string b is a subset of string a if every letter in b occurs in a including multiplicity. For example, "wrr" is a subset of "warrior" but is not a subset of "world". A string a from words1 is universal if for every string b in words2, b is a subset of a. Return an array of all the universal strings in words1. You may return the answer in any order.**

**CODE**

```
from collections import Counter
def wordSubsets(words1,words2):
    required=Counter()
    for word in words2:
        count=Counter(word)
        for ch in count:
```

```
            required[ch]=max(required[ch],count[ch])
    result=[]
    for word in words1:
        count=Counter(word)
        valid=True
        for ch in required:
            if count[ch]<required[ch]:
                valid=False
                break
        if valid:
            result.append(word)
    return result
words1=["amazon","apple","facebook","google","leetcode"]
words2=["e","o"]
print("Example 1 Output:",wordSubsets(words1,words2))
words2=["l","e"]
print("Example 2 Output:",wordSubsets(words1,words2))
```

**OUTPUT**

```
Example 1 Output: ['facebook', 'google', 'leetcode']
Example 2 Output: ['apple', 'google', 'leetcode']
```